



Paper Pigeon

An interactive 3-D map of a research ecosystem — the researchers, the labs, and the papers that quietly connect them.

Point it at any institution and watch the network draw itself.

 LIVE

[PAPER-PIGEON-T7DN.VERCEL.APP](https://paper-pigeon-t7dn.vercel.app)

 React 19  Vite 7  TypeScript 5.9  Tailwind v4 3d-force-graph WebGL  A-Frame VR

 Python 3.12  FastAPI Repopulation API  Postgres + pgvector  fly.io Deployed

 Vercel Frontend + Edge Routing

OpenAlex Works & Authors ROR Institution Resolution OpenRouter LLM + Embeddings AWS Bedrock Paper Chat

[Open the map](#) · [VR mode](#) · [Release notes](#)



See It In Action

Interactive 3-D graph

A force-directed map of researchers and labs. Drag, zoom, search, click through to profiles. No sign-in.

VR mode

Step inside the network at `/vr` — the same force layout, rendered for a headset.

On-demand discovery

Name an institution. **ROR** resolves it, **OpenAlex** supplies its researchers and works, and a scoped graph is assembled for it — query-conditioned and user-triggered, not a nightly job over a fixed corpus.

Estimated labs, real connections

Where no lab page exists, lab affiliations are inferred from co-authorship communities anchored on a likely PI. Co-authorship is fetched aligned to the discovered cohort, so a brand-new institution comes back as a connected network instead of scattered dots.

Lab-page scraping

Optionally crawls the institution's own site — robots-aware fetching, main-content extraction, then an LLM pass whose output is validated against a strict JSON schema. Anything off-schema is quarantined, never merged. Scraped text is treated as **data**, **never instructions**.

Grounded descriptions

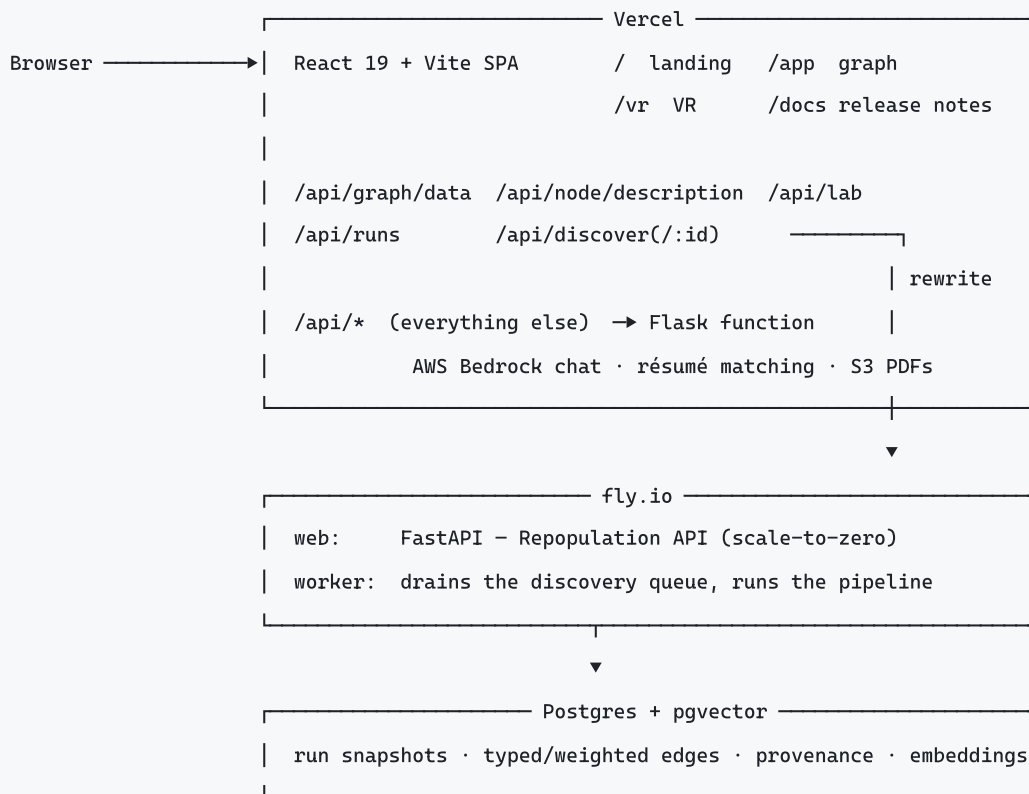
Researcher and lab blurbs are generated with the evidence that grounds them, and the citations travel with the text into the UI.

Run snapshots

Every discovery is stored as a run — each node and edge typed, weighted, and provenance-bearing. The published UW graph stays the default; **University of Toronto** and **MIT** ship as built-in examples, selectable from the run picker in the search bar.

Paper chat & resume matching

Ask questions about a paper and get cited answers; upload a résumé and get researchers worth contacting. Powered by **AWS Bedrock + S3**, and credential-gated — absent keys, the map is unaffected.



Discovery pulls from: ROR (institutions) · OpenAlex (authors, works) · institution sites (scraping)

LLM + embeddings: OpenRouter, under a DB-backed daily budget cap

How a discovery run works

1. `POST /api/discover` (key-gated) **enqueues a job** — no network work happens in the request.
2. The always-on **worker** claims it (`FOR UPDATE SKIP LOCKED`), then runs the pipeline:
ROR resolves the institution → **OpenAlex** yields authors and their works → co-authorship edges are built across the discovered cohort → labs are estimated from those communities.
3. Optionally the institution's site is **scraped** for real lab pages; extractions that fail schema validation are quarantined rather than merged.
4. Researchers and labs get **grounded descriptions** with cited evidence.
5. The result lands as a **run snapshot**, selectable from the run picker.

Every node and edge carries a `source_record_key` back to where it came from, and the loader upserts on stable identity keys — so a re-run is idempotent, not a duplicate graph.

Degrading gracefully

The published graph is a static snapshot (`public/graph_cache.json`) bundled into the API image, so the map does not depend on the database being up:

When	What happens
Postgres unreachable	<code>/api/graph/data</code> serves the bundled snapshot, marked <code>X-Graph-Source: static-cache</code>
Postgres unreachable	<code>/api/runs</code> → <code>[]</code> (the run picker hides itself); <code>grounded/lab/discovery</code> reads → <code>503</code>
The whole API is gone	The frontend falls back to the <code>/graph_cache.json</code> deployed alongside it
A specific <code>?run=</code> is requested	<code>503</code> — a run snapshot has no static equivalent, so it never silently serves a different graph

`/health` never touches the database, and connection attempts are bounded by `DATABASE_CONNECT_TIMEOUT` (default 10s) so a hanging database fails over instead of parking requests.

 Key components

Layer	Technology	Purpose
Frontend	React 19 + Vite 7 + TypeScript + Tailwind v4	SPA: landing, graph, VR, docs
3-D engine	3d-force-graph + Three.js	WebGL network visualization
VR	A-Frame + 3d-force-graph-vr	Immersive mode at <code>/vr</code>
Graph API	FastAPI on fly.io	Run snapshots, node/lab reads, discovery queue
Worker	Python process on fly.io	Drains the discovery queue, runs the pipeline
Database	Postgres + pgvector	Nodes, typed edges, runs, provenance, embeddings
Discovery sources	ROR, OpenAlex, institution websites	Institutions, researchers, works, lab pages
LLM / embeddings	OpenRouter (budget-capped)	Lab extraction, grounded descriptions
Legacy extras	Flask on Vercel + AWS Bedrock/S3	Paper chat, résumé matching, PDF access
Routing	Vercel rewrites	Graph + discovery routes proxied to fly

API

Method	Route	What it returns
GET	<code>/api/graph/data</code>	The published graph as <code>{nodes, links}</code> . <code>?run=<id></code> for a run snapshot
GET	<code>/api/runs</code>	Run snapshots available in the picker
GET	<code>/api/node/description?id=</code>	A node's grounded description + the evidence citing it
GET	<code>/api/lab?id=</code>	A lab's record: description, research areas, PI, resolved faculty
POST	<code>/api/discover</code>	Enqueue discovery of an institution — needs <code>X-Discovery-Key</code>
GET	<code>/api/discover/{job_id}</code>	Job status/stage/run id — needs <code>X-Discovery-Key</code>
GET	<code>/health</code>	Liveness. Deliberately does not touch the database

Interactive docs (Swagger) are served by the API itself at [/docs](#) on the backend.

Development

Prerequisites

```
node >= 20      # pnpm 9
python >= 3.12
```

Frontend

```
pnpm install
pnpm dev      # http://localhost:5173
```

By default the frontend calls `/api/*` on its own origin. Point it at a running backend with `VITE_API_BASE_URL=http://localhost:8000`.

Backend

```
python -m venv .venv && .venv/Scripts/activate    # macOS/Linux: source
.venv/bin/activate
pip install -r requirements-dev.txt

# Boots a no-Docker Postgres + pgvector, applies migrations, seeds the graph, serves the
API
python scripts/run_local_stack.py
```

Useful scripts: `scripts/repopulate.py` (run discovery from the CLI), `scripts/scrape_labs.py` (lab scraping), `scripts/describe.py` (grounded descriptions), `scripts/cleanup_run.py` (delete a run safely), `scripts/prod_migrate_seed.py` (migrations + seed; the fly release command).

Tests, lint, build

```
python -m pytest backend/repopulation/tests -q    # pure + pgserver integration suites
pnpm lint
pnpm build                                         # tsc -b typecheck gate + vite build
```

Environment variables

Graph API (fly.io) — the read path needs only `DATABASE_URL` :

Variable	Required	Description
<code>DATABASE_URL</code>	✓	Postgres with pgvector
<code>DATABASE_CONNECT_TIMEOUT</code>	—	Seconds to wait for a connection (default 10)
<code>DISCOVERY_API_KEY</code>	for discovery	Gates <code>POST /api/discover</code> ; fail-closed when unset
<code>OPENALEX_API_KEY</code>	—	Higher rate limits; without it the keyless polite pool is used
<code>OPENROUTER_API_KEY</code>	—	Embeddings + grounded descriptions; absent, the core graph still builds
<code>PAPERPIGEON_BUDGET_PRO_DAILY_USD</code>	—	Daily LLM spend cap, enforced in the database
<code>DISCOVERY_POLL_INTERVAL</code> / <code>DISCOVERY_IDLE_POLL_INTERVAL</code>	—	Worker poll cadence (<code>3s</code> , backing off to <code>360s</code> when the queue is idle)

Legacy AWS extras (Vercel) — optional; absent, those features simply don't appear:

Variable	Description
<code>AWS_ACCESS_KEY_ID</code> / <code>AWS_SECRET_ACCESS_KEY</code> / <code>AWS_REGION</code>	AWS credentials
<code>S3_BUCKET_NAME</code>	Bucket holding paper PDFs
<code>BEDROCK_KNOWLEDGE_BASE_ID</code>	Knowledge base for paper chat
<code>BEDROCK_KNOWLEDGE_BASE_ID_2</code>	Knowledge base for résumé matching

Deploying

```
fly deploy --app paper-pigeon-api    # API + worker; release command applies migrations
```

The frontend deploys from GitHub on merge to `main`. `vercel.json` rewrites the graph and discovery routes to fly and everything else under `/api/*` to the bundled Flask function.

Project structure

```

paper-pigeon/
├─ 📁 api/
│   └─ index.py                # Vercel serverless entry (legacy Flask app)
│
├─ 📁 backend/
│   └─ app.py                  # Flask: paper chat, résumé matching, PDFs, static cache
│   └─ 📁 controllers/         # rag · pdf · recommendations
│   └─ 📁 cache/graph_cache.json # the published UW graph
│   └─ 📁 repopulation/         # — the Repopulation Engine —
│       └─ api.py              # FastAPI service (graph reads + discovery)
│       └─ worker.py           # queue drainer: discovery → describe → scrape
│       └─ run.py, discovery/   # OpenAlex/ROR → rows, estimated labs, relevance
│       └─ scraping/, extraction/ # robots-aware fetch → clean → schema-validated LLM extraction
│       └─ descriptions/        # grounded descriptions + cited evidence
│       └─ models/, migrations/ # SQLAlchemy models; SQL migrations are the DDL of record
│       └─ tests/               # pure + pgserver integration suites
│
├─ 📁 src/
│   └─ 📁 components/
│       └─ GraphApp.tsx         # owns graph + run state for /app and /vr
│       └─ ResearchNetworkGraph.tsx, VRGraph.tsx
│       └─ SearchBar.tsx, RunSelector.tsx, DiscoveryModal.tsx
│       └─ ResearcherModal.tsx, LabModal.tsx, PaperChatModal.tsx, ...
│   └─ 📁 pages/                # landing + docs
│   └─ 📁 content/docs.ts        # release notes rendered at /docs
│   └─ 📁 services/dynamodb.ts   # frontend API client
│
├─ 📁 public/graph_cache.json    # published graph, also bundled into the API image
├─ 📁 scripts/                   # local stack, repopulate, scrape, describe, migrate+seed
├─ fly.toml, Dockerfile         # API + worker deployment
└─ vercel.json                  # frontend + API route rewrites

```

Status

This is a **beta, mid-migration**. Discovery is key-gated, the AWS extras need credentials, and the graph and discovery routes now run on FastAPI + Postgres while the original Flask function still serves the AWS-backed features. `/docs` on the live site tracks how the running system has changed, version by version.

Contributing

Pull requests welcome — for anything substantial, please open an issue first.
CI gates every PR: secret scan, frontend typecheck/build/lint, and the backend test suite.

Contact: sks17@outlook.com

Built with  for the UW Allen School research community.

TRY PAPER PIGEON NOW

